# The elephant in the interpretability room: Why use attention as explanation when we have saliency methods?

**Jasmijn Bastings**
Google Research
bastings@google.com

**Katja Filippova**
Google Research
katjaf@google.com

## Abstract

There is a recent surge of interest in using attention as explanation of model predictions, with mixed evidence on whether attention can be used as such. While attention conveniently gives us one weight per input token and is easily extracted, it is often unclear toward what goal it is used as explanation. We find that often that goal, whether explicitly stated or not, is to find out what input tokens are the most relevant to a prediction, and that the implied user for the explanation is a model developer. For this goal and user, we argue that input saliency methods are better suited, and that there are no compelling reasons to use attention, despite the coincidence that it provides a weight for each input. With this position paper, we hope to shift some of the recent focus on attention to saliency methods, and for authors to clearly state the goal and user for their explanations.

## 1 Introduction

Attention mechanisms (Bahdanau et al., 2015) have allowed for performance gains in many areas of NLP, including, *inter alia*, machine translation (Bahdanau et al., 2015; Luong et al., 2015; Vaswani et al., 2017), natural language generation (e.g., Rush et al., 2015; Narayan et al., 2018), and natural language inference (e.g., Parikh et al., 2016).

Attention has not only allowed for better performance, it also provides a window into how a model is operating. For example, for machine translation, Bahdanau et al. (2015) visualize what source tokens the target tokens are attending to, often aligning words that are translations of each other.

Whether the window that attention gives into how a model operates amounts to *explanation* has recently become subject to debate (§2). While many papers published on the topic of explainable AI have been criticised for not defining explanations (Lipton, 2018; Miller, 2019), the first key studies which spawned interest in attention as explanation (Jain and Wallace, 2019; Serrano and Smith, 2019; Wiegreffe and Pinter, 2019) do say that they are interested in whether attention weights faithfully represent the responsibility each input token has on a model prediction. That is, the narrow definition of explanation implied there is that it points at the most important input tokens for a prediction (arg max), accurately summarizing the reasoning process of the model (Jacovi and Goldberg, 2020b).

The above works have inspired some to find ways to make attention more faithful and/or plausible, by changing the nature of the hidden representations attention is computed over using special training objectives (e.g., Mohankumar et al., 2020; Tutek and Snajder, 2020). Others have proposed replacing the attention mechanism with a latent alignment model (Deng et al., 2018).

Interestingly, the implied definition of explanation in the cited works, happens to coincide with what *input saliency methods* (§3) are designed to produce (Li et al., 2016a; Sundararajan et al., 2017; Ribeiro et al., 2016; Montavon et al., 2019, i.a.). Moreover, the user of that explanation is often implied to be a model developer, to which faithfulness is important. The elephant in the room is therefore: If the goal of using attention as explanation is to assign importance weights to the input tokens in a faithful manner, why should the attention mechanism be preferred over the multitude of existing input saliency methods designed to do *exactly that*? In this position paper, with that goal in mind, we argue that we should pay attention no heed (§4). We propose that we reduce our focus on attention as explanation, and shift it to input saliency methods instead. However, we do emphasize that understanding the *role* of attention is still a valid research goal (§5), and finally, we discuss a few approaches that go beyond saliency (§6).



*Proceedings of the Third BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP*, pages 149–155
Online, November 20, 2020. ©2020 Association for Computational Linguistics

## 2 The Attention Debate

In this section we summarize the debate on whether attention is explanation. The debate mostly features simple BiLSTM text classifiers (see Figure 1). Unlike Transformers (Vaswani et al., 2017), they only contain a single attention mechanism, which is typically MLP-based (Bahdanau et al., 2015):

$$e_i = \mathbf{v}^\top tanh(W_h \mathbf{h}_i + W_q \mathbf{q}) \quad \alpha_i = \frac{\exp e_i}{\sum_k \exp e_k} \quad (1)$$

where $\alpha_i$ is the attention score for BiLSTM state $\mathbf{h}_i$. When there is a single input text, there is no query, and $\mathbf{q}$ is either a trained parameter (like $\mathbf{v}$, $W_h$ and $W_q$), or $W_q\mathbf{q}$ is simply left out of Eq. 1.

### 2.1 Is attention (not) explanation?

Jain and Wallace (2019) show that attention is often uncorrelated with gradient-based feature importance measures, and that one can often find a completely different set of attention weights that results in the same prediction. In addition to that, Serrano and Smith (2019) find, by modifying attention weights, that they often do not identify those representations that are most most important to the prediction of the model. However, Wiegreffe and Pinter (2019) claim that these works do not disprove the usefulness of attention as explanation *per se*, and provide four tests to determine if or when it can be used as such. In one such test, they are able to find alternative attention weights using an adversarial training setup, which suggests attention is not always a faithful explanation. Finally, Pruthi et al. (2020) propose a method to produce deceptive attention weights. Their method reduces how much weight is assigned to a set of 'impermissible' tokens, even when the models demonstratively rely on those tokens for their predictions.

### 2.2 Was the right task analyzed?

In the attention-as-explanation research to date text classification with LSTMs received the most scrutiny. However, Vashishth et al. (2019) question why one should focus on single-sequence tasks at all because the attention mechanism is arguably far less important there than in models involving two sequences, like NLI or MT. Indeed, the performance of an NMT model degrades substantially if uniform weights are used, while random attention weights affect the text classification performance minimally. Therefore, findings from text classification studies may not generalize to tasks where attention is a crucial component.

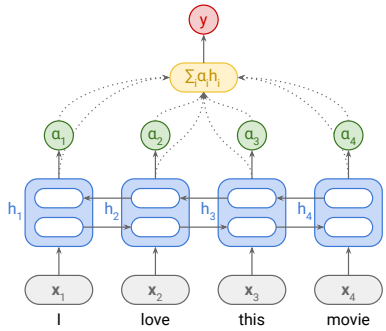


Figure 1: A typical model in the debate.

Interestingly, even for the task of MT, the first case where attention was visualized to inspect a model (§1), Ding et al. (2019) find that saliency methods (§3) yield better word alignments.

### 2.3 Is a causal definition assumed?

Grimsley et al. (2020) go as far as saying that attention is not explanation by definition, if a causal definition of explanation is assumed. Drawing on the work in philosophy, they point out that causal explanations presuppose that a surgical intervention is possible which is not the case with deep neural networks: one cannot intervene on attention while keeping all the other variables invariant.

### 2.4 Can attention be improved?

The problems with using as attention as explanation, especially regarding faithfulness, have inspired some to try and 'improve' the attention weights, so to make them more faithful and/or plausible. Mohankumar et al. (2020) observe high similarity between the hidden representations of LSTM states and propose a diversity-driven training objective that makes the hidden representations more diverse across time steps. They show using representation erasure that the resulting attention weights result in decision flips more easily as compared to vanilla attention. With a similar motivation, Tutek and Snajder (2020) use a word-level objective to achieve a stronger connection between hidden states and the words they represent, which affects attention. Not part of the recent debate, Deng et al. (2018) propose variational attention as an alternative to the soft attention of Bahdanau et al. (2015), arguing that the latter is not *alignment*, only an approximation thereof. They have the additional benefit of allowing posterior alignments, conditioned on the input and the output sentences.

## 3 Saliency Methods

In this section we discuss various input saliency methods for NLP as alternatives to attention: gradient-based (§3.1), propagation-based (§3.2), and occlusion-based methods (§3.3), following Arras et al. (2019). We do not endorse any specific method[1], but rather try to give an overview of methods and how they differ. We discuss methods that are applicable to *any* neural NLP model, allowing access to model internals, such as activations and gradients, as attention itself requires such access. We leave out more expensive methods that use a surrogate model, e.g., LIME (Ribeiro et al., 2016).

### 3.1 Gradient-based methods

While used earlier in other fields, Li et al. (2016a) use gradients as explanation in NLP and compute:

$$\nabla_{\mathbf{x}_i} f_c(\mathbf{x}_{1:n}) \tag{2}$$

where $\mathbf{x}_i$ is the input word embedding for time step $i$, $\mathbf{x}_{1:n} = \langle \mathbf{x}_1, \ldots, \mathbf{x}_n \rangle$ are the input embeddings (e.g., a sentence), and $f_c(\mathbf{x}_{1:n})$ the model output for target class $c$. After taking the L2 norm of Eq. 2, the result is a measure of how sensitive the model is to the input at time step $i$.

If instead we take the dot product of Eq. 2 with the input word embedding $\mathbf{x}_i$, we arrive at the gradient×input method (Denil et al., 2015), which returns a saliency (scalar) of input $i$:

$$\nabla_{\mathbf{x}_i} f_c(\mathbf{x}_{1:n}) \cdot \mathbf{x}_i \tag{3}$$

Integrated gradients (IG) (Sundararajan et al., 2017) is a gradient-based method which deals with the problem of *saturation*: gradients may get close to zero for a well-fitted function. IG requires a baseline $\mathbf{b}_{1:n}$, e.g., all-zeros vectors or repeated [MASK] vectors. For input $i$, we compute:

$$\frac{1}{m}\sum_{k=1}^{m} \nabla_{\mathbf{x}_i} f_c\Big(\mathbf{b}_{1:n} + \frac{k}{m}(\mathbf{x}_{1:n} - \mathbf{b}_{1:n})\Big) \cdot (\mathbf{x}_i - \mathbf{b}_i) \tag{4}$$

That is, we average over $m$ gradients, with the inputs to $f_c$ being linearly interpolated between the baseline and the original input $\mathbf{x}_{1:n}$ in $m$ steps. We then take the dot product of that averaged gradient with the input embedding $\mathbf{x}_i$ minus the baseline.

We propose distinguishing *sensitivity* from *saliency*, following Ancona et al. (2019): the former says how much a change in the input changes the output, while the latter is the marginal effect of each input word on the prediction. Gradients measure sensitivity, whereas gradient×input and IG measure saliency. A model can be sensitive to the input at a time step, but it depends on the actual input vector if it was important for the prediction.

### 3.2 Propagation-based methods

Propagation-based methods (Landecker et al., 2013; Bach et al., 2015; Arras et al., 2017, i.a.), of which we discuss Layer-wise Relevance Propagation (LRP) in particular, start with a forward pass to obtain the output $f_c(\mathbf{x}_{1:n})$, which is the top-level *relevance*. They then use a special backward pass that, at each layer, *redistributes* the incoming relevance among the inputs of that layer. Each kind of layer has its own propagation rules. For example, there are different rules for feed-forward layers (Bach et al., 2015) and LSTM layers (Arras et al., 2017). Relevance is redistributed until we arrive at the input layers. While LRP requires implementing a custom backward pass, it does allow precise control to preserve relevance, and it has been shown to work better than using gradient-based methods on text classification (Arras et al., 2019).

### 3.3 Occlusion-based methods

Occlusion-based methods (Zeiler and Fergus, 2014; Li et al., 2016b) compute input saliency by occluding (or erasing) input features and measuring how that affects the model. Intuitively, erasing unimportant features does not affect the model, whereas the opposite is true for important features. Li et al. (2016b) erase word embedding dimensions and whole words to see how doing so affects the model. They compute the importance of a word *on a dataset* by averaging over how much, for each example, erasing that word caused a difference in the output compared to not erasing that word.

As a saliency method, however, we can apply their method on a single example only. For input $i$:

$$f_c(\mathbf{x}_{1:n}) - f_c(\mathbf{x}_{1:n|\mathbf{x}_i=0}) \tag{5}$$

computes saliency, where $\mathbf{x}_{1:n|\mathbf{x}_i=0}$ indicates that input word embedding $\mathbf{x}_i$ was zeroed out, while the other inputs were unmodified. Kádár et al. (2017) and Poerner et al. (2018) use a variant, *omission*, by simply leaving the word out of the input.

This method requires $n+1$ forward passes. It is also used for evaluation, to see if important words another method has identified bring a change in model output (e.g., DeYoung et al., 2020).

[1] For an evaluation of methods for explaining LSTM-based models, see e.g., Poerner et al. (2018) and Arras et al. (2019).